Python Style Guide:

Данный документ представляет собой подробную выжимку правил и стандартов программирования на языке Python, основанных на кодстайле Google. Он базируется на PEP 8, но вводит более строгие ограничения для обеспечения масштабируемости и читаемости кода в крупных проектах.

1. Форматирование кода и визуальный стиль

Стиль форматирования направлен на то, чтобы код выглядел единообразно во всех модулях системы.

- **Длина строки:** Лимит составляет **90 символов**. Исключения допускаются только для длинных URL, путей импорта или длинных сообщений об ошибках в комментариях.
- **Отступы:** Используйте ровно **4 пробела (Tab)** для каждого уровня вложенности.
- **Перенос строк:** Предпочтительным способом является использование естественного группирования в скобках (), [] и {}.

Правильно:

```
def long_function_name(
        var_one, var_two, var_three,
        var_four):
    print(var_one)
```

Неправильно (использование обратного слеша):

```
if width > 10 and \
   height > 20:
    pass
```

- **Пустые строки:** Одна пустая строка между методами класса или функциями.
- **Кавычки:** Используйте либо ', либо " последовательно внутри одного файла. Для строк документации (docstrings) всегда используются """тройные двойные кавычки""".

2. Организация импортов

Основные правила:

1. **Импорт модулей:** Импортируйте только пакеты или модули. Не импортируйте конкретные функции или классы (за исключением пакетов typing и collections.abc).
2. **Один импорт — одна строка:** Каждый модуль должен быть на отдельной строке.

Тип импорта	Пример (Правильно ✅)	Пример (Неправильно ❌)
Модули	import os	import os, sys

Тип импорта	Пример (Правильно ✅)	Пример (Неправильно ❌)
	import sys	
Функции	import math val = math.sqrt(4)	from math import sqrt val = sqrt(4)

3. Соглашения об именовании

Имена должны быть информативными. Избегайте сокращений, которые понятны только вам.

- **Пакеты и модули:** module_name (нижний регистр, snake_case).
- **Классы:** ClassName (PascalCase).
- **Функции и методы:** function_name() (snake_case).
- **Переменные:** variable_name (snake_case).
- **Константы:** CONSTANT_NAME (верхний регистр, SCREAMING_SNAKE_CASE).
- **Внутренние элементы:** Начинаются с одного подчеркивания (_private_method).

4. Документирование (Docstrings)

Каждая публичная функция, класс или модуль должны иметь docstring в формате Google Style. Можно писать как на английском, так и на русском языках.

```
def calculate_area(radius: float) -> float:
    """Вычисляет площадь круга по его радиусу.

    Args:
        radius: Радиус круга (положительное число).

    Returns:
        Площадь круга.

    Raises:
        ValueError: Если радиус меньше нуля.
    """
    if radius < 0:
        raise ValueError("Радиус не может быть отрицательным")
    return 3.14159 * (radius ** 2)
```

5. Лучшие практики и особенности языка

Аргументы по умолчанию

Никогда не используйте изменяемые объекты (списки, словари) в качестве значений по умолчанию в определении функции.

Правильно:

```
def add_item(item, target_list=None):
    if target_list is None:
        target_list = []
    target_list.append(item)
    return target_list
```

Неправильно (список создается один раз при компиляции функции):

```
def add_item(item, target_list=[]):
    target_list.append(item)
    return target_list
```

Проверки на истинность (True/False)

Используйте лаконичные проверки для пустых контейнеров, но будьте внимательны с None.

Правильно (для списков/строк):

```
if not my_list:
    print("Список пуст")
```

Правильно (явная проверка на None):

```
if value is not None:
    # Делаем что-то с значением, которое может быть нулем
    pass
```

Типизация (Type Hinting)

Аннотации типов обязательны для нового кода (не относится к выходам функций). Они помогают инструментам статического анализа и улучшают читаемость.

```
def process_user_data(name: str, age: int):
    return {"user": name, "age": age}
```

Исключения

- Никогда не используйте "голый" except:. Всегда указывайте конкретный тип ошибки.